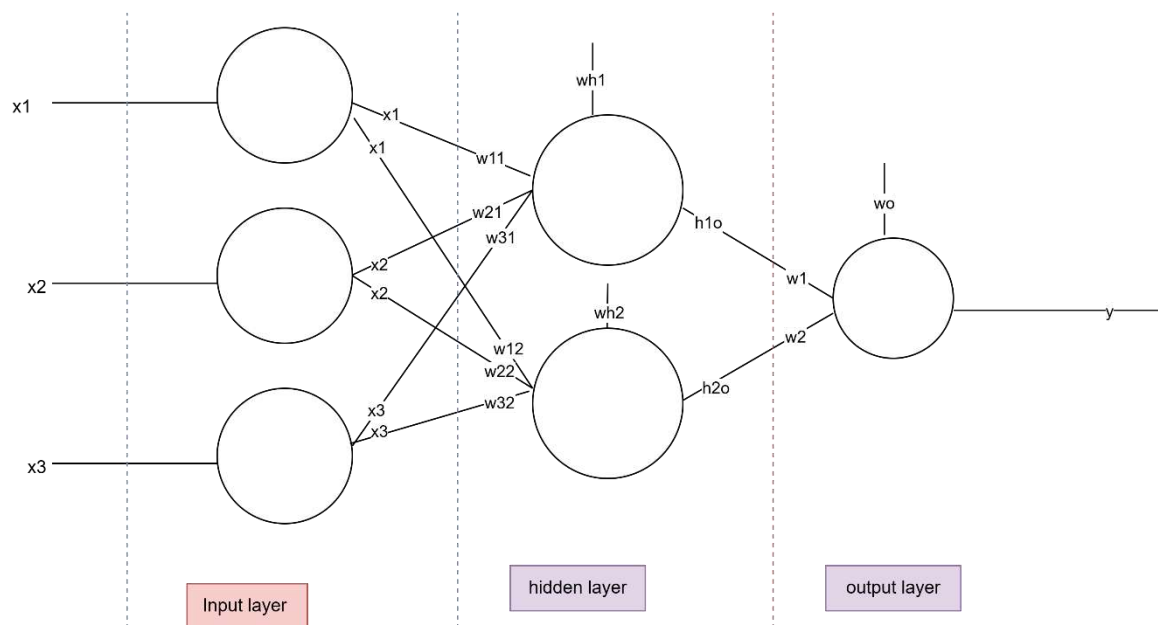


Deep learning
How to find the parameters

1. We are able to draw the NN
2. We are able to write math equations
3. we are able to write Matrix format
4. We are able to Calculate number of Parameters : weights + bias

Qns yourslef

- q1) How to find the parameters*
- q2) How to speed up the process (Optimization methods)*
- q3) How many actiavtion functions are there, in which layer which activation function need to use*
- q4) How many hidden layer choose , How to avoid overfitting (Regularization methods)*



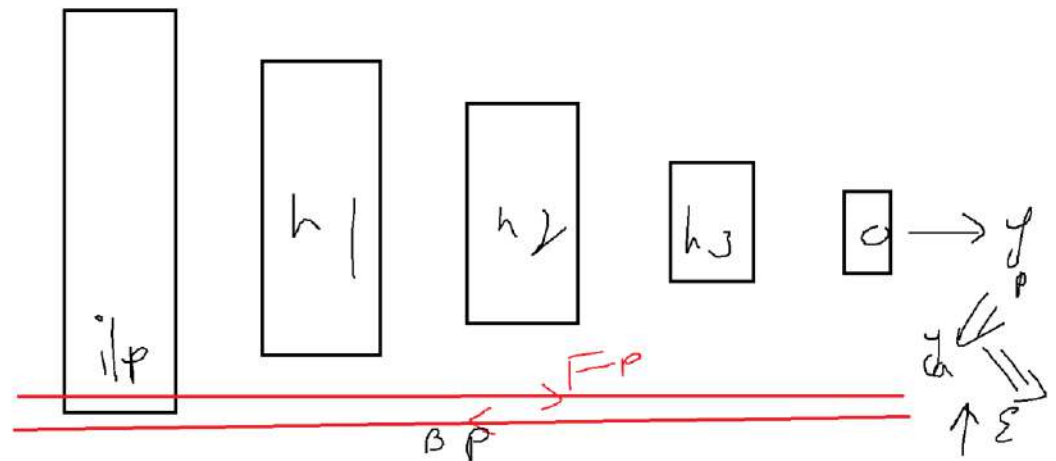
forward prpogation

- In the above NN x_1, x_2, x_3 are inputs
- the first hidden layer parameter : $w_{11}, w_{12}, w_{21}, w_{22}, w_{31}, w_{32}, w_{h1}, w_{h2}$
- These parameter we provide as a **random value initially**
- like all the layer parameters provided randomly
- now the computations process will continue till last output layer
- finally we will get the final predictions = y_{pred}
- in the above process the computations from input layer to output layer will process this method is called as **forward prpogation**

the predictions will compare with y_{actual} and we will calculate error
generally these error is very huge because the correct output depends on paramters

the same anlogy with Linear regression, th main aim in LR is minimise the cost function by using best parameters or coeff

but we provided coeff or parameters randomly , naturally the error become huge



Back prpogation

- The error cause due to all layers coefficients or parameters
- This is called as **credit assignment**
- So again NN travel from Output layer to input layer to update the weights
- This process is called as **Back propagation**

Epoch

- *In Forward propagation computations happens and $y_{prediction}$ will come*
- *In back propagation the weights will upade in order to reduce the errors*
- *This entire process is called as Epoch*
- *in simply Epoch means one iteration*

Epoch = Forward prpogation + Backward Prpogation

all the training samples should complete then it consider as one epoch

we need to repeat this process till we will get error constant

if we give more and more epochs there might be a chance model

become overfit

How to find the parameters

- *OLS*
- *Gradient Descent Method*

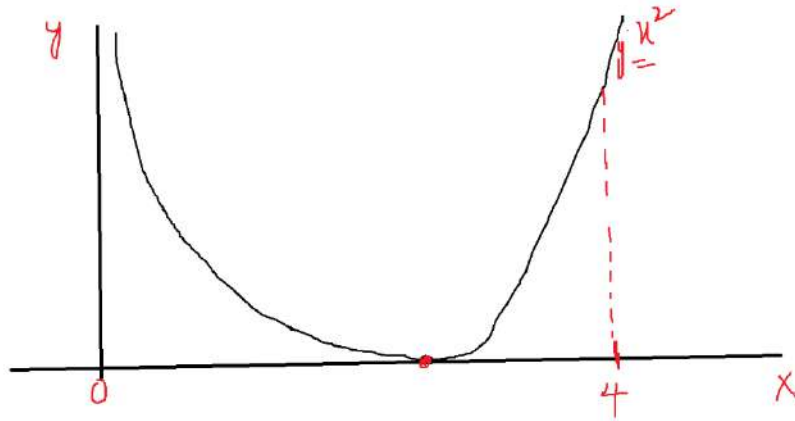
Gradient means difference

Gradient means slope

Gradient descent means decrease the slope

Decrease the slope means we are reaching a minmum point

Decrease the difference measn we are reduceing the change



in above diagram the main goal is to find the minimum point
we don't know exact minimum point

we are assuming the equation $y = x^2$ has a minimum point at $x = 4$

Thumb rule:

at mini point any equation the slope becomes zero

Step – 1: find the slope of the equation $y = x^2$

Step – 2:

substitute $x = 4$, if slope y' becomes zero then it has min point at $x = 4$

otherwise it is wrong

$$y = x^2$$

$$\frac{dy}{dx} = y' = 2x$$

$$y' \text{ at } 4 = 2(4) = 8$$

8 means slope is there, so at $x = 4$ it is not a minimum point

how can achieve my minimum point?

should we increase the x or decrease the x

Rule :

if previous slope is positive then decrease the value of x

if previous slope is negative increase the value of x

$$X_{new} = X_{old} - \text{distance} \quad (\text{use only numbers with out sign})$$

distance traveled = how much slope we got previously

$$X_{new} = X_{old} - \frac{dy}{dx} \Big|_{x=x_{old}}$$

$$X_{old} = 4$$

$$\frac{dy}{dx} \Big|_{x=x_{old}} = 2x \text{ at } x = 4 = 8$$

$$X_{new} = 4 - 8 = -4$$

$$X_{new} = -4$$

now repeat the same process

$$y = x^2$$

$$y' = 2x$$

$$x = -4 \text{ then } y' = 2(-4) = -8 \text{ not equal to zero } 0$$

$$X_{new} = X_{old} - \text{alpha} * \frac{dy}{dx} \Big|_{x=x_{old}}$$

- *alpha = learning rate the values ranges from 0 to 1*
- *0 means no learning*
- *1 fast learning*
- *both are not useful*
- *in real time we will change the alpha 0.2 to 0.5*

$$\alpha = 0.2$$

$$X_{new} = 4 - 0.2 * 8 = 4 - 1.6 = 2.4$$

$$X_{new} = 2.4$$

$$w_{new} = w_{old} - \alpha * \frac{\partial J}{\partial w} \Big|_{w=w_{old}}$$

$$w_{o_{new}} = w_{o_{old}} - \alpha * \frac{\partial J}{\partial w_o} \Big|_{w_o=w_{old}}$$

